

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: MLA0302

COURSE CODE: Reinforcement learning

COURSE OUTCOME COVERED

CO1: *Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 10

Total Marks: 50

Annexure A

RL Basic Experiments demo with Keras and TensorFlow using Anaconda navigator

Duration: 2hrs

Experiment 1: Installation of Reinforcement Learning Development Environment

Aim: To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium (OpenAI Gym), NumPy, Matplotlib, and other required libraries, and verify the environment by executing a simple Reinforcement Learning program.

Experiment 2: Familiarization with OpenAI Gym/Gymnasium Environments

Aim: To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole, and MountainCar using Gymnasium to understand the concepts of states, actions, rewards, and episodes.

Experiment 3: Implementation of the Reinforcement Learning Framework

Aim: To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through states, actions, rewards, and policies using Python.

Experiment 4: Simulation of a Markov Decision Process (MDP)

Aim: To develop a simple Markov Decision Process model and simulate state transitions, reward functions, and policy execution for understanding sequential decision-making problems.

Experiment 5: Bellman Equation Demonstration

Aim: To implement Bellman Expectation and Bellman Optimality Equations for calculating state-value and action-value functions and analyze the convergence of value estimation.

Experiment 6: Exploration vs. Exploitation Strategies

Aim: To implement ϵ -Greedy, Greedy, and Random action-selection strategies and compare their performance in balancing exploration and exploitation during Reinforcement Learning.

Experiment 7: Multi-Armed Bandit Problem

Aim: To implement the Multi-Armed Bandit problem using ϵ -Greedy and Upper Confidence Bound (UCB) algorithms and evaluate cumulative rewards obtained by different action-selection methods.

Experiment 8: Reward Visualization in Reinforcement Learning

Aim: To simulate an RL agent in a simple environment and visualize cumulative rewards, episode rewards, and learning performance using Matplotlib graphs.

Experiment 9: TensorFlow and Keras-Based Neural Network for RL

Aim: To build a simple feed-forward neural network using TensorFlow and Keras for approximating value functions in Reinforcement Learning environments.

Experiment 10: Mini Reinforcement Learning Project Using CartPole

Aim: To develop a basic Reinforcement Learning agent using TensorFlow and Keras to solve the CartPole environment and evaluate its learning performance through episode rewards and success rate.

Code of Conduct:

I _____ M Poojitha Reddy__192472352_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

M. Poojitha Reddy

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

EXPERIMENT 1

CODE :

```
import gymnasium as gym
env = gym.make("CartPole-v1", render_mode="human")

observation, info = env.reset()

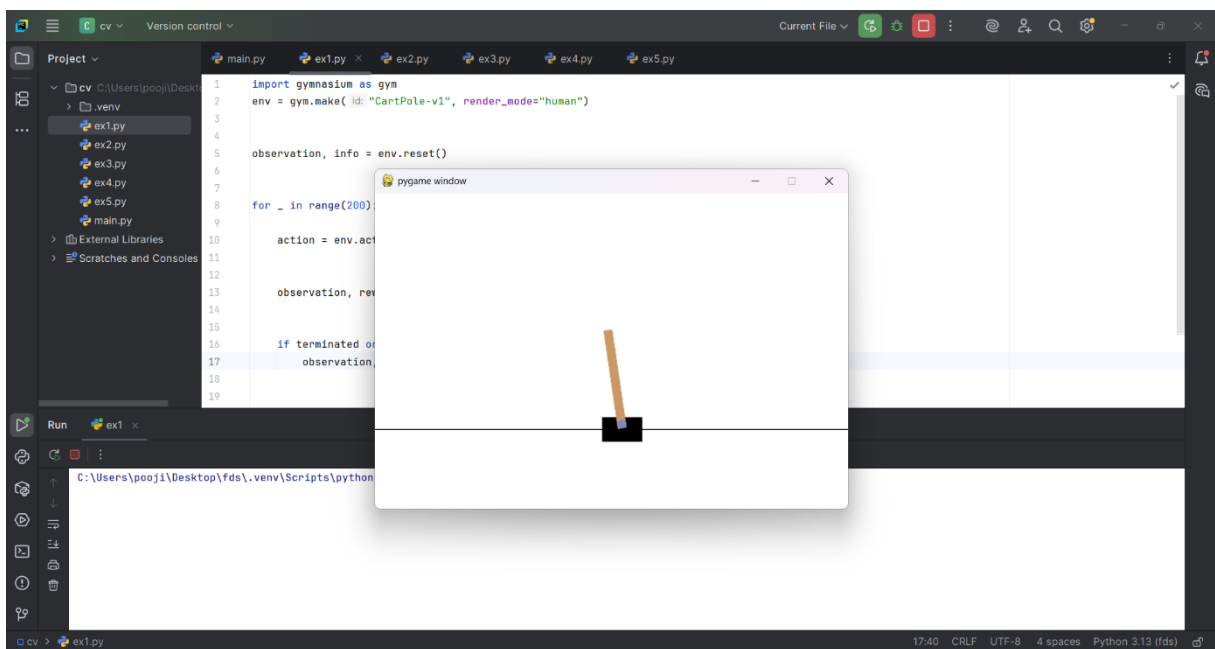
for _ in range(200):

    action = env.action_space.sample()

    observation, reward, terminated, truncated, info = env.step(action)

    if terminated or truncated:
        observation, info = env.reset()
```

OUTPUT :



EXPERIMENT 2

CODE :

```
import gymnasium as gym
```

```
while True:
```

```
    print("\n--- Gymnasium Environments ---")
    print("1. FrozenLake")
    print("2. CartPole")
    print("3. MountainCar")
    print("4. Exit")
```

```
    choice = int(input("Enter your choice: "))
```

```
    if choice == 1:
```

```
        env = gym.make("FrozenLake-v1")
        env_name = "FrozenLake"
```

```
    elif choice == 2:
```

```
        env = gym.make("CartPole-v1")
        env_name = "CartPole"
```

```
    elif choice == 3:
```

```
        env = gym.make("MountainCar-v0")
        env_name = "MountainCar"
```

```
    elif choice == 4:
```

```
        print("Program Ended")
        break
```

```
    else:
```

```
        print("Invalid Choice")
        continue
```

```
    print("\nSelected Environment:", env_name)
```

```
    state, info = env.reset()
```

```
    print("Initial State:", state)
```

```
    for step in range(5):
```

```
        action = env.action_space.sample()
        next_state, reward, terminated, truncated, info = env.step(action)
```

```
        print(f"\nStep {step + 1}")
```

```
        print("Action:", action)
```

```
        print("Next State:", next_state)
```

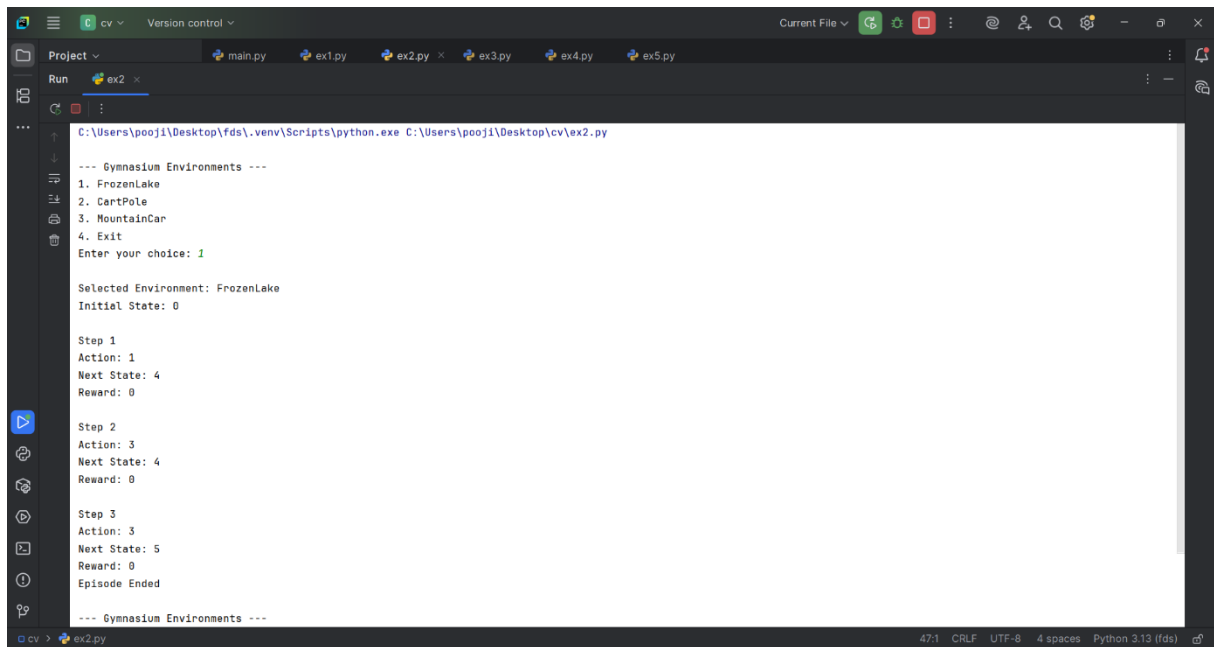
```
        print("Reward:", reward)
```

```
state = next_state
```

```
if terminated or truncated:  
    print("Episode Ended")  
    break
```

```
env.close()
```

OUTPUT :



```
C:\Users\pooji\Desktop\fds\.venv\Scripts\python.exe C:\Users\pooji\Desktop\cv\ex2.py  
  
--- Gymnasium Environments ---  
1. FrozenLake  
2. CartPole  
3. MountainCar  
4. Exit  
Enter your choice: 1  
  
Selected Environment: FrozenLake  
Initial State: 0  
  
Step 1  
Action: 1  
Next State: 4  
Reward: 0  
  
Step 2  
Action: 3  
Next State: 4  
Reward: 0  
  
Step 3  
Action: 3  
Next State: 5  
Reward: 0  
Episode Ended  
  
--- Gymnasium Environments ---
```

EXPERIMENT 3

CODE :

```
import gymnasium as gym
```

```
# Create the environment
```

```
env = gym.make("FrozenLake-v1", is_slippery=False)
```

```
# Start a new episode
```

```
state, info = env.reset()
```

```
print("Initial State:", state)
```

```
# Run one episode
```

```
for step in range(10):
```

```
# Agent selects a random action (policy)
```

```
action = env.action_space.sample()
```

```

# Environment executes the action
next_state, reward, terminated, truncated, info = env.step(action)

print(f"Step {step + 1}")
print("Current State:", state)
print("Action:", action)
print("Next State:", next_state)
print("Reward:", reward)
print()

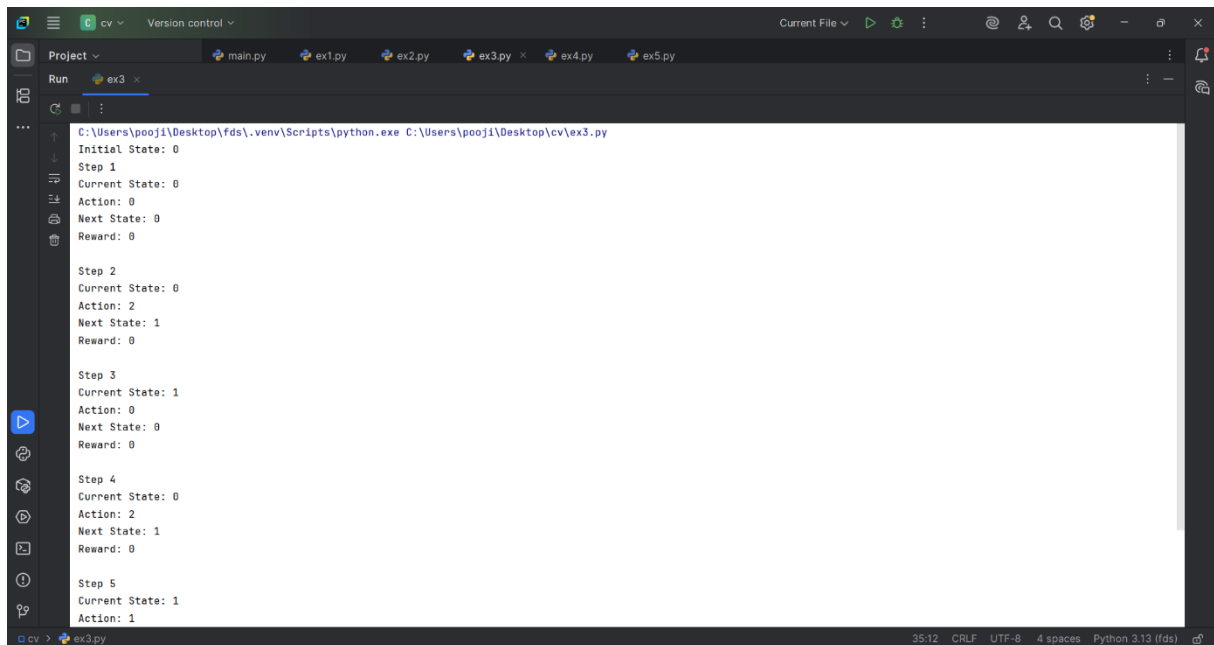
# Update the current state
state = next_state

# Stop if the episode ends
if terminated or truncated:
    print("Episode Ended")
    break

env.close()

```

OUTPUT :



```

C:\Users\pooj1\Desktop\fds\.venv\Scripts\python.exe C:\Users\pooj1\Desktop\cv\ex3.py
Initial State: 0
Step 1
Current State: 0
Action: 0
Next State: 0
Reward: 0

Step 2
Current State: 0
Action: 2
Next State: 1
Reward: 0

Step 3
Current State: 1
Action: 0
Next State: 0
Reward: 0

Step 4
Current State: 0
Action: 2
Next State: 1
Reward: 0

Step 5
Current State: 1
Action: 1

```

EXPERIMENT 4

CODE :

```
# States
states = ["A", "B", "Goal"]

# Reward function
rewards = {
    ("A", "right"): 0,
    ("B", "right"): 10
}

# State transition function
transitions = {
    ("A", "right"): "B",
    ("B", "right"): "Goal"
}

# Policy
policy = {
    "A": "right",
    "B": "right"
}

# Initial state
state = "A"

print("Initial State:", state)

# Execute policy
while state != "Goal":

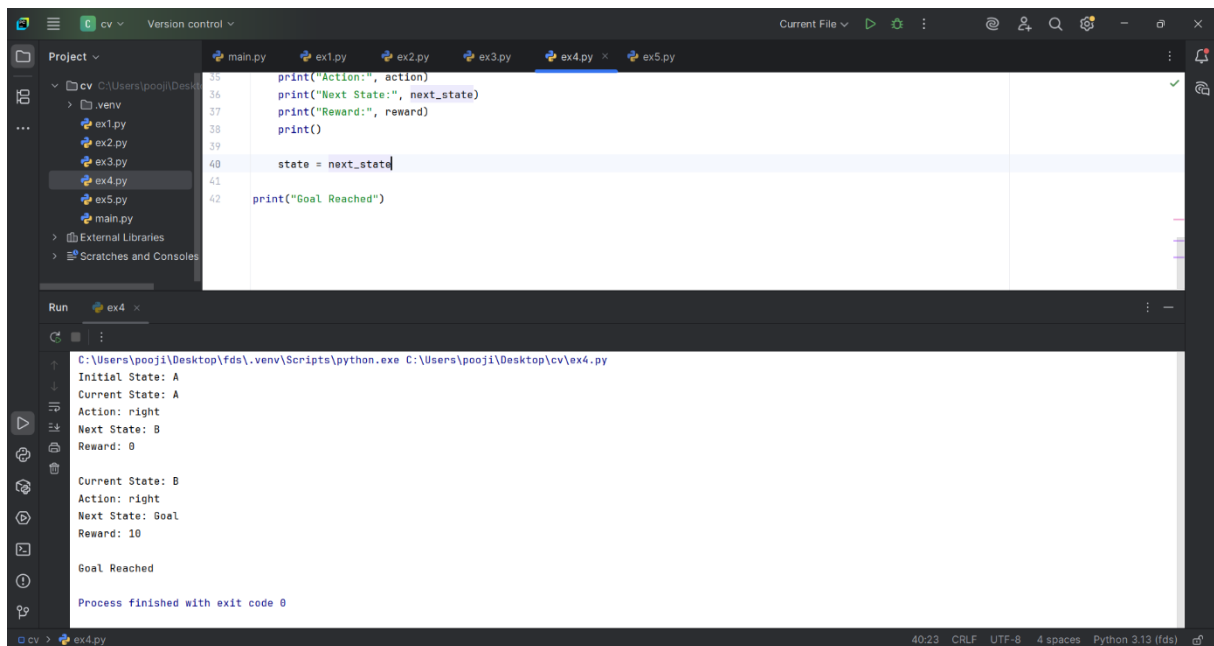
    action = policy[state]
    next_state = transitions[(state, action)]
    reward = rewards[(state, action)]

    print("Current State:", state)
    print("Action:", action)
    print("Next State:", next_state)
    print("Reward:", reward)
    print()

    state = next_state

print("Goal Reached")
```


OUTPUT :



The screenshot shows a code editor with a project named 'cv'. The file explorer on the left shows a directory structure with files 'ex1.py', 'ex2.py', 'ex3.py', 'ex4.py', 'ex5.py', and 'main.py'. The editor is open to 'ex4.py', which contains the following Python code:

```
35 print("Action:", action)
36 print("Next State:", next_state)
37 print("Reward:", reward)
38 print()
39
40 state = next_state
41
42 print("Goal Reached")
```

The 'Run' panel at the bottom shows the output of the code execution:

```
C:\Users\pooji\Desktop\fds\.venv\Scripts\python.exe C:\Users\pooji\Desktop\cv\ex4.py
Initial State: A
Current State: A
Action: right
Next State: B
Reward: 0

Current State: B
Action: right
Next State: Goal
Reward: 10

Goal Reached
Process finished with exit code 0
```

EXPERIMENT 5

CODE :

```
import numpy as np
```

```
# States in Grid World
```

```
states = [0, 1, 2, 3]
```

```
# Actions
```

```
actions = ["Left", "Right"]
```

```
# Reward for reaching goal state
```

```
rewards = {
    (0, "Right"): 0,
    (1, "Right"): 0,
    (2, "Right"): 10
}
```

```
# Transition function
```

```
transitions = {
    (0, "Right"): 1,
```

```

    (1, "Right"): 2,
    (2, "Right"): 3
}

# Discount factor
gamma = 0.9

# Initial state values
V = np.zeros(len(states))

# Policy: always choose Right
policy = {
    0: "Right",
    1: "Right",
    2: "Right"
}

# Bellman Expectation Equation
print("Bellman Expectation Updates")

for iteration in range(5):

    new_V = np.zeros(len(states))

    for state in range(3):

        action = policy[state]

        reward = rewards[(state, action)]

        next_state = transitions[(state, action)]

        new_V[state] = reward + gamma * V[next_state]

    V = new_V

    print("Iteration", iteration + 1, "Values:", V)

# Bellman Optimality Equation

Q = np.zeros((len(states), len(actions)))

print("\nBellman Optimality Action Values")

for state in range(3):

    for action_index, action in enumerate(actions):

        if (state, action) in rewards:

```

```

reward = rewards[(state, action)]

next_state = transitions[(state, action)]

Q[state][action_index] = reward + gamma * np.max(Q[next_state])

print(Q)

print("\nOptimal State Values:")
print(np.max(Q, axis=1))

```

OUTPUT :

```

76 print(Q)
77
78 print("\nOptimal State Values:")
79 print(np.max(Q, axis=1))

```

Run ex5

```

C:\Users\poofj\Desktop\fds\.venv\Scripts\python.exe C:\Users\poofj\Desktop\cv\ex5.py
Bellman Expectation Updates
Iteration 1 Values: [ 0.  0. 10.  0.]
Iteration 2 Values: [ 0.  9. 10.  0.]
Iteration 3 Values: [ 8.1  9. 10.  0.]
Iteration 4 Values: [ 8.1  9. 10.  0.]
Iteration 5 Values: [ 8.1  9. 10.  0.]

Bellman Optimality Action Values
[[ 0.  0.]
 [ 0.  0.]
 [ 0. 10.]
 [ 0.  0.]]

Optimal State Values:
[ 0.  0. 10.  0.]

Process finished with exit code 0

```

EXPERIMENT 6

CODE :

```

import numpy as np
import matplotlib.pyplot as plt

true_rewards = [2, 5, 3, 7, 4]
episodes = 100

def run(strategy):
    q = np.zeros(5)
    count = np.zeros(5)
    rewards = []

```

```

for i in range(epochs):

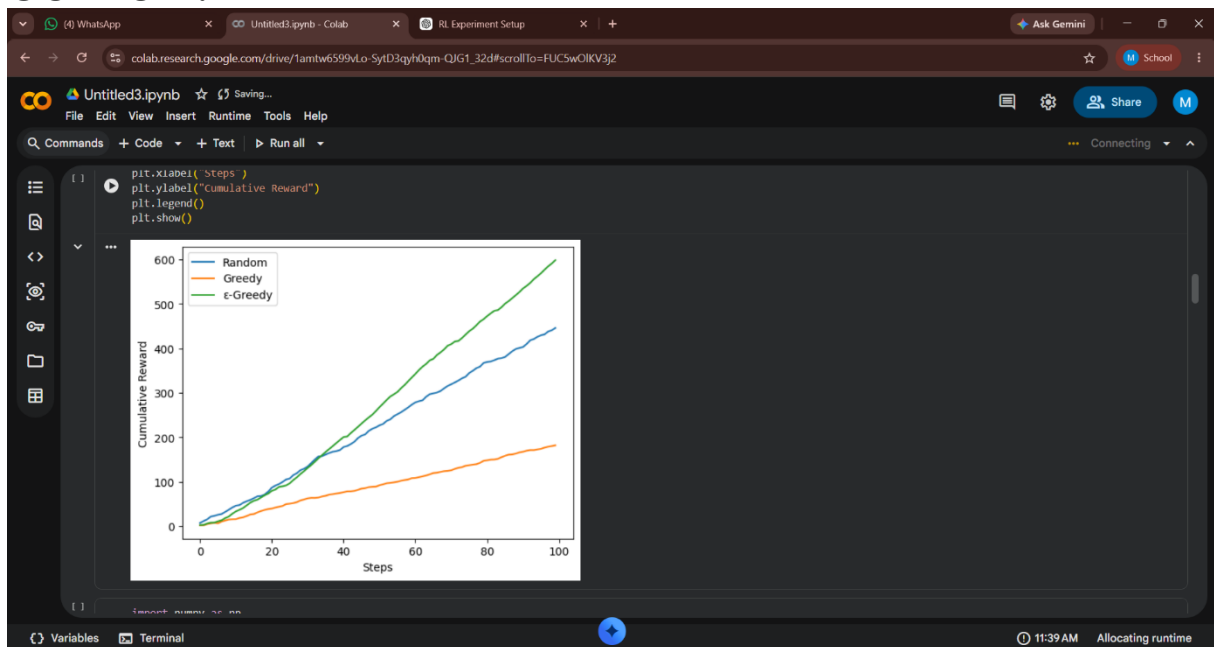
    if strategy == "random":
        action = np.random.randint(5)

    elif strategy == "greedy":
        action = np.argmax(q)

    else:
        # epsilon-greedy
        if np.random.rand() < 0.1:
            action = np.ra

```

OUTPUT :



EXPERIMENT 7

CODE :

```

import numpy as np
import matplotlib.pyplot as plt
arms = [1,3,5,7,9]
steps = 500
def epsilon_greedy():
    q = np.zeros(5)
    n = np.zeros(5)
    rewards = []
    for t in range(steps):
        if np.random.rand() < 0.1:
            a = np.random.randint(5)
        else:

```

```

        a = np.argmax(q)

        r = np.random.normal(arms[a],1)
        n[a]+=1
        q[a] += (r-q[a])/n[a]
        rewards.append(r)

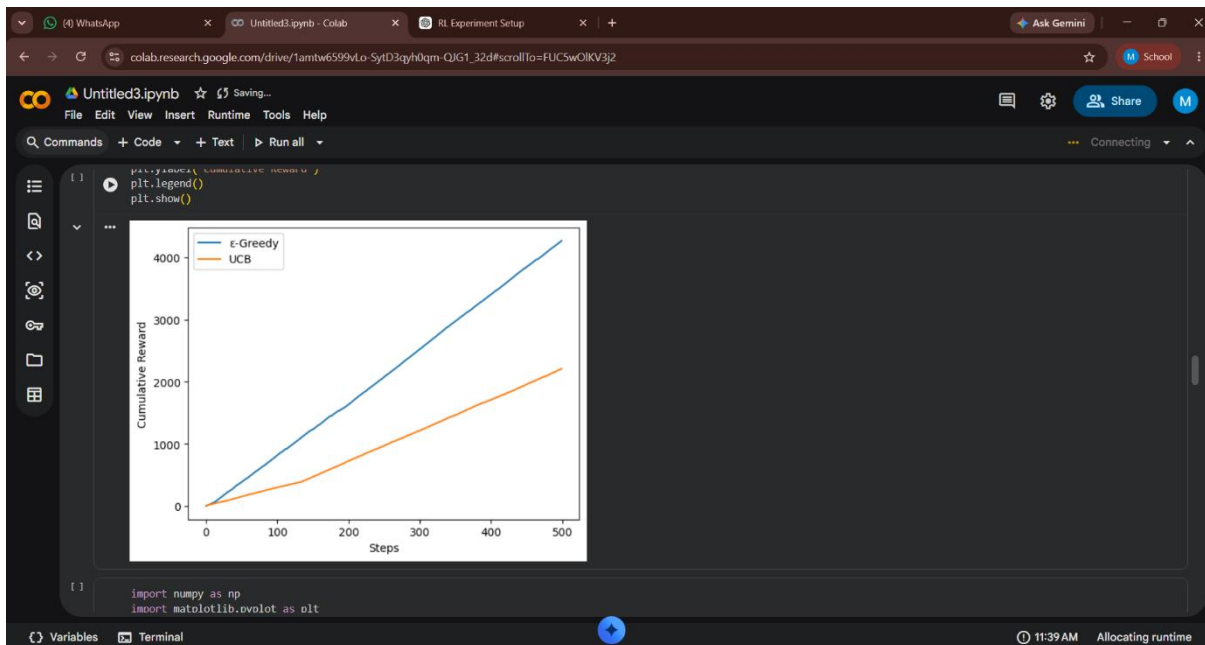
    return np.cumsum(rewards)
def ucb():
    q=np.zeros(5)
    n=np.ones(5)
    rewards=[]
    for t in range(5):
        rewards.append(np.random.normal(arms[t],1))
    for t in range(5,steps):
        u=q+np.sqrt(2*np.log(t+1)/n)
        a=np.argmax(u)

        r=np.random.normal(arms[a],1)
        n[a]+=1
        q[a] += (r-q[a])/n[a]
        rewards.append(r)

    return np.cumsum(rewards)
plt.plot(epsilon_greedy(),label="ε-Greedy")
plt.plot(ucb(),label="UCB")
plt.xlabel("Steps")
plt.ylabel("Cumulative Reward")
plt.legend()
plt.show()

```

OUTPUT :



EXPERIMENT 8

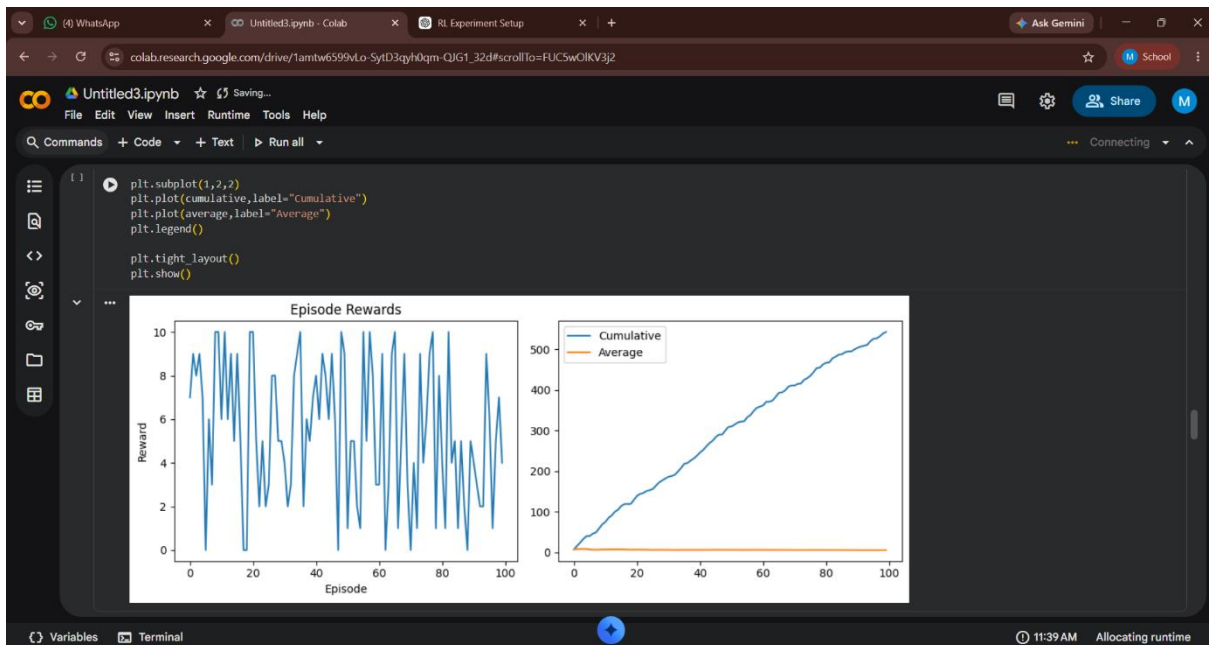
CODE :

```
import numpy as np
import matplotlib.pyplot as plt
episodes = 100
episode_rewards = []
for i in range(episodes):
    reward = np.random.randint(0,11)
    episode_rewards.append(reward)
cumulative = np.cumsum(episode_rewards)
average = cumulative/(np.arange(episodes)+1)
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.plot(episode_rewards)
plt.title("Episode Rewards")
plt.xlabel("Episode")
plt.ylabel("Reward")

plt.subplot(1,2,2)
plt.plot(cumulative,label="Cumulative")
plt.plot(average,label="Average")
plt.legend()

plt.tight_layout()
plt.show()
```

OUTPUT :



EXPERIMENT 9

CODE :

```
import tensorflow as tf
import numpy as np

model = tf.keras.Sequential([
    tf.keras.layers.Dense(24,activation='relu',input_shape=(4,)),
    tf.keras.layers.Dense(24,activation='relu'),
    tf.keras.layers.Dense(2)
])

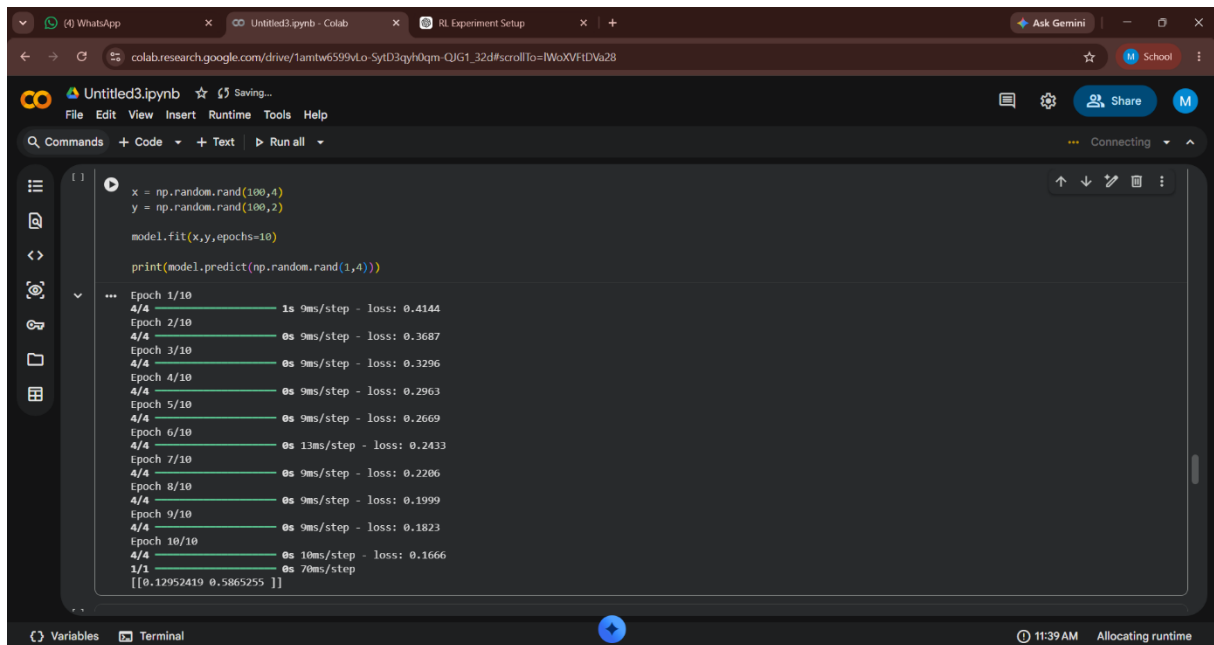
model.compile(optimizer='adam',
              loss='mse')

x = np.random.rand(100,4)
y = np.random.rand(100,2)

model.fit(x,y,epochs=10)

print(model.predict(np.random.rand(1,4)))
```

OUTPUT :



```
x = np.random.rand(100,4)
y = np.random.rand(100,2)

model.fit(x,y,epochs=10)

print(model.predict(np.random.rand(1,4)))
```

Epoch 1/10 1s 9ms/step - loss: 0.4144
4/4
Epoch 2/10 0s 9ms/step - loss: 0.3687
4/4
Epoch 3/10 0s 9ms/step - loss: 0.3296
4/4
Epoch 4/10 0s 9ms/step - loss: 0.2963
4/4
Epoch 5/10 0s 9ms/step - loss: 0.2669
4/4
Epoch 6/10 0s 13ms/step - loss: 0.2433
4/4
Epoch 7/10 0s 9ms/step - loss: 0.2206
4/4
Epoch 8/10 0s 9ms/step - loss: 0.1999
4/4
Epoch 9/10 0s 9ms/step - loss: 0.1823
4/4
Epoch 10/10 0s 10ms/step - loss: 0.1666
2/2 0s 70ms/step
[[0.12952419 0.5865255]]

EXPERIMENT -10

CODE :

```
import gymnasium as gym
import tensorflow as tf
import numpy as np
import random

env = gym.make("CartPole-v1")

model = tf.keras.Sequential([
    tf.keras.layers.Dense(24,activation="relu",input_shape=(4,)),
    tf.keras.layers.Dense(24,activation="relu"),
    tf.keras.layers.Dense(2)
])

model.compile(optimizer="adam",loss="mse")

gamma = 0.95
epsilon = 1.0

memory = []

episodes = 100

for episode in range(episodes):

    state,_ = env.reset()
```



```

state = np.reshape(state,[1,4])

total_reward = 0
done = False

while not done:

    if np.random.rand() < epsilon:
        action = env.action_space.sample()
    else:
        action = np.argmax(model.predict(state,verbose=0)[0])

    next_state,reward,terminated,truncated,_ = env.step(action)
    done = terminated or truncated

    next_state = np.reshape(next_state,[1,4])

    memory.append((state,action,reward,next_state,done))

    state = next_state
    total_reward += reward

    if len(memory) > 32:

        batch = random.sample(memory,32)

        for s,a,r,ns,d in batch:

            target = model.predict(s,verbose=0)

            if d:
                target[0][a] = r
            else:
                future = np.max(model.predict(ns,verbose=0))
                target[0][a] = r + gamma*future

            model.fit(s,target,epochs=1,verbose=0)

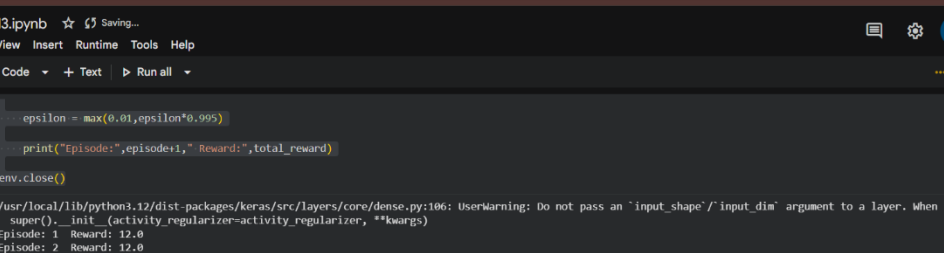
    epsilon = max(0.01,epsilon*0.995)

    print("Episode:",episode+1," Reward:",total_reward)

env.close()

```

OUTPUT :



The screenshot displays a Jupyter Notebook interface with the following components:

- Top Bar:** Shows the notebook title "Untitled3.ipynb" and the environment "Colab".
- File Menu:** Includes options like File, Edit, View, Insert, Runtime, Tools, and Help.
- Code Cell:** Contains the following Python code:

```
epsilon = max(0.01, epsilon*0.995)

print("Episode:", episode+1, "Reward:", total_reward)

env.close()
```
- Output:** Displays a warning message from Keras and a list of 21 episodes with their corresponding rewards:

```
Warning: Do not pass an 'input_shape' / 'input_dim' argument to a layer. When using Sequential
Episode: 1 Reward: 12.0
Episode: 2 Reward: 12.0
Episode: 3 Reward: 39.0
Episode: 4 Reward: 42.0
Episode: 5 Reward: 36.0
Episode: 6 Reward: 10.0
Episode: 7 Reward: 16.0
Episode: 8 Reward: 24.0
Episode: 9 Reward: 16.0
Episode: 10 Reward: 13.0
Episode: 11 Reward: 21.0
Episode: 12 Reward: 10.0
Episode: 13 Reward: 14.0
Episode: 14 Reward: 35.0
Episode: 15 Reward: 26.0
Episode: 16 Reward: 11.0
Episode: 17 Reward: 10.0
Episode: 18 Reward: 11.0
Episode: 19 Reward: 21.0
Episode: 20 Reward: 32.0
Episode: 21 Reward: 40.0
```
- Bottom Bar:** Shows the status "Allocating runtime" and the time "11:39 AM".

